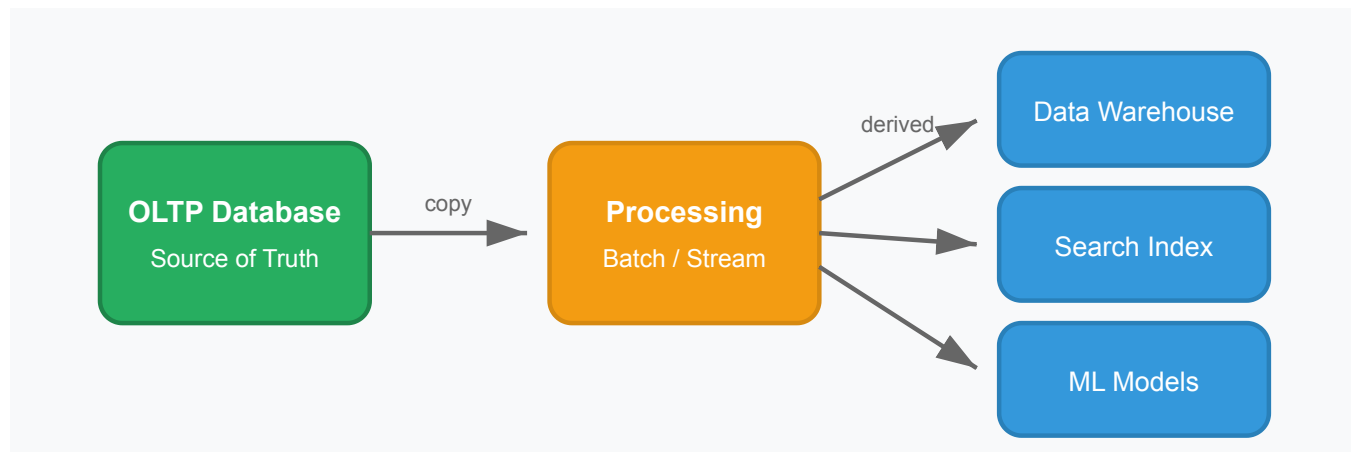


Overview and When to Use Each

 systemdesignschool.io/fundamentals/batch-and-stream-processing



Batch & Stream Processing: Overview

Your application database handles individual requests well. A user loads their profile—the query returns a few rows in milliseconds. But what happens when someone asks a question that requires scanning everything?

Consider a retail company analyzing yesterday's sales. The query needs every transaction from every store—millions of rows. Running this on the production database creates problems: the query holds locks for minutes, consumes CPU needed for checkout traffic, and competes with real-time orders. At scale, this analytical query could slow down or even crash the system serving customers.

This is why production systems separate transaction processing from analytical processing. Let's understand why, and how the two processing approaches—batch and stream—address different needs.

OLTP vs OLAP: Two Different Access Patterns

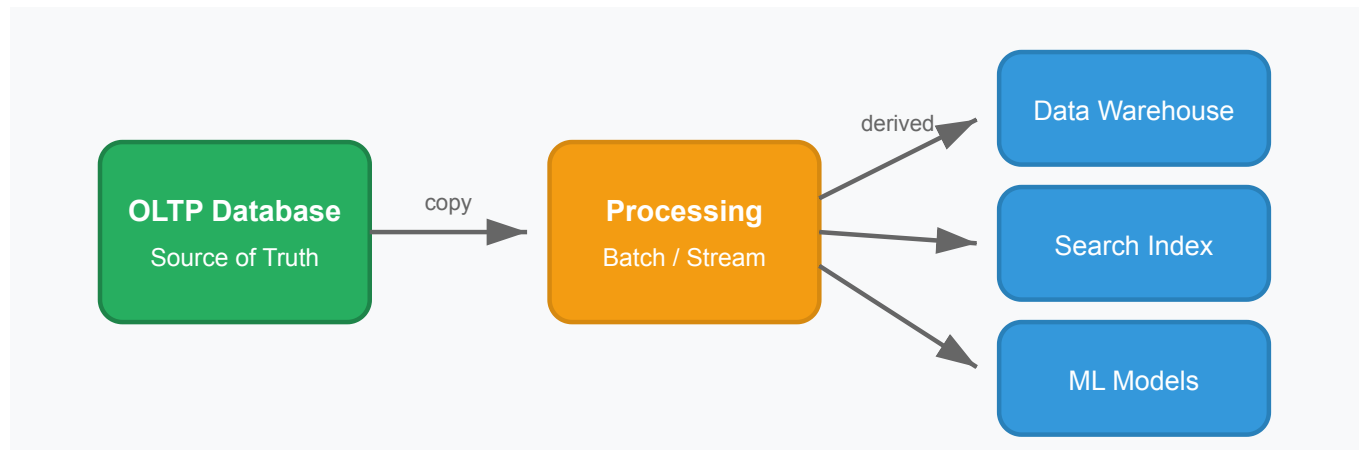
Your production database handles **OLTP (Online Transaction Processing)**—many small, fast operations targeting specific rows. Insert an order. Update inventory. Check a balance. Each query touches a handful of rows using indexed lookups.

Business intelligence questions look different. "What's our best-selling product by region this quarter?" doesn't target specific rows. It scans millions of transactions, groups them by region and product, and aggregates totals. This is **OLAP (Online Analytical Processing)**.

Why can't we run OLAP queries on OLTP databases? The database isn't designed for it. OLTP databases optimize for quick row lookups using B-tree indexes. But analytical queries ignore indexes—they need full table scans. A long-running scan holds locks, fills the buffer pool with

analytical data (evicting hot transactional data), and competes with user requests for CPU.

The solution: copy data to a separate system designed for analysis. The OLTP database stays fast for users. The analytical system handles expensive computations.



Derived Data

Your application database is the **source of truth**—it holds the authoritative version of user accounts, orders, and inventory. Everything else is **derived data**: transformed copies optimized for specific purposes.

Derived data is expendable. If you lose it, rebuild from the source. Examples include:

- A search index built from product records
- Daily sales summaries aggregated from transactions
- A recommendation model trained on user behavior
- A denormalized cache of frequently accessed data

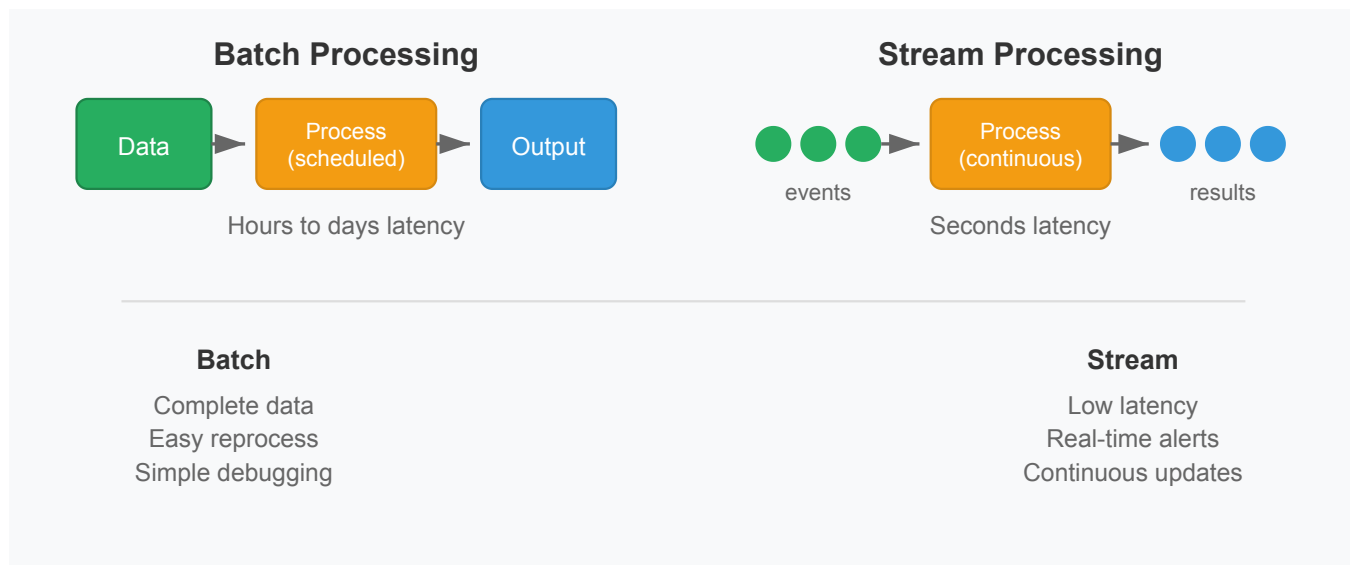
Here's the insight that makes large-scale data processing work: derived data lets you run expensive computations without touching production databases. Copy the data. Process it elsewhere. Rebuild derived outputs whenever the source changes. Users never notice because the OLTP database handles their requests normally.

Two Processing Approaches

Now let's explore how we actually create derived data. There are two fundamental approaches.

Batch processing handles data in scheduled chunks. You run a job every hour or every night. The job processes all accumulated data at once. Results appear after the job completes—latency measured in minutes to hours.

Stream processing handles data continuously. Each event is processed as it arrives. Results appear within seconds. The processing never "finishes"—it runs continuously as new data flows in.



Think of it this way: batch processing is like doing laundry once a week. You accumulate dirty clothes, wash them all at once, and have clean clothes afterward. Stream processing is like having a magical washing machine that cleans each item the moment you drop it in.

Choosing Between Batch and Stream

Neither approach is universally better—they optimize for different constraints.

Factor	Batch	Stream
Latency tolerance	Hours to days	Seconds to minutes
Data completeness	Full historical data	Recent events only
Reprocessing	Easy—rerun the job	Complex—need replay infrastructure
Debugging	Simpler—examine frozen dataset	Harder—data keeps flowing

Batch fits when:

- Results don't need to be immediate (daily reports, weekly summaries)
- You need the complete dataset to produce correct results (historical trends, ML training)
- Debugging and reprocessing matter more than speed

Stream fits when:

- Delays cause business harm (fraud detection must block transactions in real-time)
- Users expect live updates (dashboards, notifications)
- Data arrives continuously and you want continuous outputs

Many systems use both. A retailer might use stream processing to detect fraudulent transactions in real-time, while using batch processing to generate daily sales reports and train recommendation models. The same source data flows through both pipelines, each optimized for its use case.

What's Ahead

This section builds your understanding of data processing from foundations to production patterns:

Batch Processing starts with Unix pipelines—the conceptual foundation that influenced MapReduce. We'll cover MapReduce's map/shuffle/reduce phases, HDFS for distributed storage, and Spark's improvements over MapReduce.

Stream Processing explains how to handle unbounded data: windowing strategies, the distinction between event time and processing time, and delivery guarantees (at-most-once, at-least-once, exactly-once).

Hybrid Architectures compares Lambda architecture (batch + stream layers) and Kappa architecture (stream-only), then covers modern unified frameworks.

Data Pipelines addresses practical concerns: ETL vs ELT patterns, backfill strategies when you need to reprocess historical data, and error handling with dead-letter queues.

Real-Time Analytics shows how to build live dashboards using materialized views and time-series patterns.

From here, Unix pipelines provide the conceptual model that shaped how we think about data processing at scale.